Lab 5 Files names permissions and storage on Linux

Tacu Igor, FAF 242
Operating Systems, Technical University of Moldova
Run on 9 November 2026

Purpose and required execution identity

The experiment connects pathnames, inodes, permissions and disk allocation using the original Lab 5 commands. Every command here ran on the Ubuntu x86_64 server as `oslab_igor`, UID 1001, without sudo or extra groups. An ordinary user's failures demonstrate permission checks that root may bypass. The complete recording is `Lab5-6/evidence/linux.txt`; the command sequence is `scripts/run-lab5.sh`.

```
+ whoami
oslab_igor
+ id
uid=1001(oslab_igor) gid=1001(oslab_igor) groups=1001(oslab_igor)
+ pwd
/tmp/igor-lab5.KuLBB2
```

Part 1 The tree and file types

The first letters in ls identified `/etc` and `/tmp` as directories (`d`), `/etc/passwd` as a regular file (`-`), `/bin` as a symbolic link (`l`), and `/dev/null` as a character device (`c`). `/bin` resolves to `/usr/bin`. File recognized ls as an ELF executable and passwd as text. A character device was `/dev/zfs`; a block device was `/dev/sr0`.

The accessible recursive totals were 476 MiB for `/var/log` and 3.5 GiB for `/usr`, making usr approximately seven to eight times larger. As requested, the du commands suppress permission diagnostics; inaccessible entries can make an ordinary user's total incomplete. These values describe this server at the time of the run.

```
+ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx   1 root root     7 Apr 22  2024 /bin -> usr/bin
crw-rw-rw-   1 root root  1, 3 Sep 28 13:52 /dev/null
drwxr-xr-x 119 root root 12288 Nov  9 14:10 /etc
-rw-r--r--   1 root root  1849 Nov  9 14:10 /etc/passwd
drwxrwxrwt  46 root root  4096 Nov  9 14:10 /tmp
+ ls -l /dev
+ head
total 0
crw-r--r-- 1 root root     10, 235 Sep 28 13:52 autofs
drwxr-xr-x 2 root root         300 Sep 28 13:52 block
drwxr-xr-x 2 root root          80 Sep 28 13:52 bsg
crw-rw---- 1 root disk     10, 234 Sep 28 13:52 btrfs-control
drwxr-xr-x 3 root root          60 Sep 28 13:52 bus
lrwxrwxrwx 1 root root           3 Sep 28 13:52 cdrom -> sr0
drwxr-xr-x 2 root root        3460 Sep 28 14:28 char
crw--w---- 1 root tty       5,   1 Sep 28 13:52 console
lrwxrwxrwx 1 root root          11 Sep 28 13:52 core -> /proc/kcore
+ file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,
    interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=44900829
    b39271d878615772bb2a6ffa422f0891, for GNU/Linux 3.2.0, stripped
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin
+ readlink -f /bin
/usr/bin
+ find /dev -maxdepth 1 -type c -print
+ head -1
/dev/zfs
+ find /dev -maxdepth 1 -type b -print
+ head -1
/dev/sr0
+ du -sh /var/log
476M    /var/log
+ du -sh /usr
3.5G    /usr
```

Part 2 Inodes links and allocation

Stat showed inode 3113 and Links 1 for notes.txt. The inode number identifies the object; Links counts directory names referring to it. After ln, notes.txt and hard.txt both reported inode 3113 and two links. Removing notes.txt deleted one directory entry, leaving the data accessible through hard.txt with one link.

Soft.txt had a separate inode and an eight-byte length, exactly the length of its stored relative target `hard.txt`. After that target was deleted, soft.txt still existed but cat failed to resolve it. A symlink stores a pathname, not an additional hard link to the target inode.

```
+ ln notes.txt hard.txt
+ ls -li notes.txt hard.txt
3113 -rw-r--r-- 2 oslab_igor oslab_igor 6 Nov  9 14:10 hard.txt
3113 -rw-r--r-- 2 oslab_igor oslab_igor 6 Nov  9 14:10 notes.txt
+ rm notes.txt
+ cat hard.txt
hello
+ ls -li hard.txt
3113 -rw-r--r-- 1 oslab_igor oslab_igor 6 Nov  9 14:10 hard.txt
+ ln -s hard.txt soft.txt
+ ls -li hard.txt soft.txt
3113 -rw-r--r-- 1 oslab_igor oslab_igor 6 Nov  9 14:10 hard.txt
3115 lrwxrwxrwx 1 oslab_igor oslab_igor 8 Nov  9 14:10 soft.txt -> hard.txt
+ cat soft.txt
hello
+ rm hard.txt
+ cat soft.txt
cat: soft.txt: No such file or directory
+ printf 'broken symlink cat status=%d\n' 1
broken symlink cat status=1
+ ls -l soft.txt
lrwxrwxrwx 1 oslab_igor oslab_igor 8 Nov  9 14:10 soft.txt -> hard.txt
```

A new directory had two links: its entry in the parent and its own dot entry. The dot-dot entries in two new child directories added two references, making the parent's link count four. The two-byte one.txt consumed a 4 KiB allocation unit. Sparse big.img had logical length 1 GiB but no allocated data blocks, so du returned zero. File length and storage consumption measure different things.

Part 3 Permissions

The requested modes are 600 = `rw-------` for a private file; 644 = `rw-r--r--` for an ordinary shared-readable document; 755 = `rwxr-xr-x` for a public executable or traversable directory; and 700 = `rwx------` for a private executable or directory. Ownership selects the owner permission class; it does not override zero owner-read bits. Cat therefore failed at mode 000 even though the user owned s.txt. Restoring owner-read permission made the file readable.

Umask was 0022. Default file mode 0666 masked by 0022 became 0644, while default directory mode 0777 became 0755. This is bit masking: the mask removes requested permissions and cannot add execute bits to an ordinary new file.

```
+ chmod 600 s.txt
+ ls -l s.txt
-rw------- 1 oslab_igor oslab_igor 7 Nov  9 14:10 s.txt
+ chmod 000 s.txt
+ cat s.txt
cat: s.txt: Permission denied
+ printf 'mode 000 cat status=%d\n' 1
mode 000 cat status=1
+ chmod u+r s.txt
+ cat s.txt
secret
+ umask
0022
+ touch new.txt
+ mkdir newdir
+ ls -ld new.txt newdir
-rw-r--r-- 1 oslab_igor oslab_igor    0 Nov  9 14:10 new.txt
drwxr-xr-x 2 oslab_igor oslab_igor 4096 Nov  9 14:10 newdir
+ mkdir box
+ echo hi
+ chmod u=r,go= box
+ ls box
f
+ cat box/f
cat: box/f: Permission denied
+ printf 'read-only directory cat status=%d\n' 1
read-only directory cat status=1
+ chmod u=x,go= box
+ ls box
ls: cannot open directory 'box': Permission denied
+ printf 'execute-only directory ls status=%d\n' 2
execute-only directory ls status=2
+ cat box/f
hi
+ chmod 755 box
```

With only read permission on box, ls could read the name f but cat could not traverse the directory to open box/f. With only execute permission, ls could not list names but cat could access the already-known pathname. Directory r permits reading entries, while x permits searching and traversing them. These outcomes were actually observed under the unprivileged identity.

The s on `/usr/bin/passwd` is set-user-ID: its controlled code can execute with the file owner’s effective identity to update protected password information. The t on `/tmp` is the sticky bit, restricting removal of other users’ entries in a shared writable directory. Reading `/etc/shadow` was denied to oslab_igor, consistent with root/shadow ownership and its restricted mode. No password hashes were recorded.

Part 4 Disk diagram and proc

The disk diagram below shows sda, approximately 38.1 GiB, with sda1 as the ext4 root filesystem, sda14 as a 1 MiB boot-related partition without a displayed mount, and sda15 as a 256 MiB vfat EFI filesystem mounted at `/boot/efi`. Sr0 is a separate optical device. Root usage was 38 GiB total, 14 GiB used, 22 GiB available and 39 percent use; rounding and filesystem reservations explain why displayed columns need not sum exactly.

```
+ lsblk -o NAME,TYPE,SIZE,FSTYPE,MOUNTPOINTS
NAME    TYPE  SIZE FSTYPE MOUNTPOINTS
sda     disk 38.1G
|-sda1  part 37.9G ext4   /
|-sda14 part    1M
`-sda15 part  256M vfat   /boot/efi
sr0     rom  1024M
+ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        38G   14G   22G  39% /
+ findmnt -no SOURCE,FSTYPE,TARGET /
/dev/sda1 ext4   /
```

Proc reported two processor entries and MemTotal 3,911,572 KiB, approximately 3.73 GiB. MemAvailable is an observation of the current workload, not installed capacity. Reading `/proc/self/status` with head reported Name head because self resolves for the process reading it. Likewise the fd listing described ls's own descriptors, including its open directory descriptor 3. Descriptors 1 and 2 point to the saved recording in this run because output was redirected there.

Conclusion

The outputs distinguish names from inode identity, symlink contents from target existence, and logical length from allocated space. The non-root identity makes the permission failures meaningful. Disk and proc values are observations of the actual Ubuntu server. Optional stretch exercises are not claimed as completed.

Execution captures and code excerpts

The native Terminal screenshot below was supplied by the author from a direct unprivileged permission test. The earlier full raw execution recording remains in the repository. This additional live check uses a separate temporary directory.

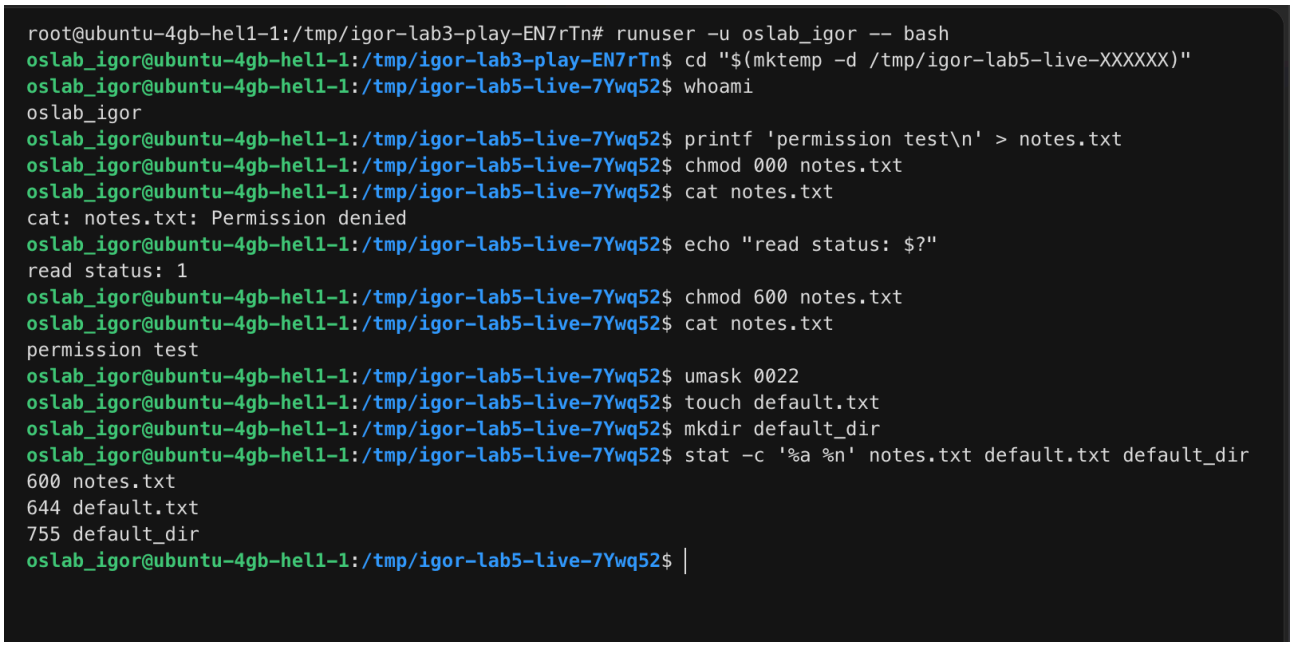


Figure 1: Unprivileged permission checks